TEMCO LEARN

Temco Platform

`temco-ui` · built Nov 10, 2026

`temco-ui`

The application's shared UI conventions, and nothing else.

Folder	`extensions/temco-ui`
Extension id	`temcocontrols.temco-ui`
Namespace	`temcoUi`
State	active
Surface	none — no command, no view, no setting
Requires	nothing
Required by	every module that opens a page (`temco-repositories` , `temco-docs` , …)

Responsibilities, one file each

Module	Owns
`src/extension.ts`	activation, and the API below
`src/columns.ts`	`pageColumn()` and `fileColumn()` — the one tab group our pages open in, and the same group for the files they open beside them
`src/rows.ts`	`rowSlug()` — what a command argument names: a slug, a row, or a row's own command argument
`src/page.ts`	`page()` — the document around a page, `escapeHtml()` , and the click-to-command script a page runs
`media/page.css`	the page vocabulary: what a module draws inside a page tab

The page vocabulary

The mock site (`docs/mock`) is where this application's pages were designed, and `media/page.css` is that design shipped to the product: the same class names and shapes as `docs/mock/css/views.css` ,

so a page can be designed in the mock and built here without being re-designed.

What changes between the two hosts is only colour. The mock owns literal colours; the product reads the theme the person chose. The top block of `page.css` is that whole difference — every mock colour role (`--fg` , `--border` , `--accent` , `--ok` …) is mapped onto a `--vscode-*` variable, and every rule after it means the same thing in both places. Sizes follow `--vscode-font-size` , so raising the editor font size makes a page larger rather than ignored.

`page(body, { title, script })` returns a complete document: the stylesheet inlined (a webview that links a file needs `localResourceRoots` and a URI that survives the server case; a page needs neither), a content-security policy, and the body inside `.view > .view__inner` , which is the width rule the mock already uses.

A page runs a command by message, not by link. Anything carrying `data-command` posts `{ type: 'command', command, args }` back to its module when clicked; `commandScript` is that script and `runCommand(message)` is the half that runs it. A page therefore never decides what a command means, and a module can audit the whole of what its pages can do in one place. It also means `enableScripts` buys exactly one behaviour instead of a small application per page.

The column rule, and why it is a rule

Our pages are tabs in one group. The dashboard, a project, the AI page, the repositories page and the documentation pages they open share a single tab strip, so whichever one is in front fills the editor area. Never a split: a page in half the window is not "shown", and two groups means one of them is always too narrow.

The rule was settled on 2026-10-14 after two wrong answers, both of which asked "is the tab in front one of ours?" and fell back to `Beside` — which is what produced the split, because a webview tab does not report its view type the way that check assumed. It asks nothing about what is in front now: it finds the group that already holds a page of ours and opens there, so our pages can only ever occupy one group, whatever is clicked, from wherever.

The prefix it matches is `temco` , so every module's view type counts as one of ours — `temcoDocs.*` , `temcoAi.page` , `temcoRepositories.page` .

Why a module and not a helper inside one of the others

The rule lived in `temco-docs/src/surface.ts` , which was right while every page was that module's. Once pages spread across modules, the rule had to belong to something *none* of them owns: a page opened from `temco-ai` and a page opened from `temco-repositories` must land in the same strip, and a copy of the rule per module is a copy that will diverge — which is exactly how the split it exists to prevent came back the first two times.

Consumers

Declare it in `extensionDependencies` and read it through a bridge of your own — `extensions/temco-repositories/src/ui.ts` is the copy to follow — because an extension cannot import another extension's sources.